

Working with APIs (Updated) (CodeGrade)

FORK

[_ \(https://github.com/learn-co-curriculum/phase-4v2-python/fork\)](https://github.com/learn-co-curriculum/phase-4v2-python/fork)  [_ \(https://github.com/learn-co-curriculum/phase-4v2-python\)](https://github.com/learn-co-curriculum/phase-4v2-python) 
[_ \(https://github.com/learn-co-curriculum/phase-4v2-python/issues/new\)](https://github.com/learn-co-curriculum/phase-4v2-python/issues/new)

Learning Goals

- Learn some of the basic ways to work with APIs.

Key Vocab

- **Request:** an attempt by one machine to contact another over the internet.
- **Client:** an application or machine that accesses services being provided by a server through the internet.
- **Web Server:** a combination of software and hardware that uses Hypertext Transfer Protocol (HTTP) and other protocols to respond to requests made over the internet.

Define API

An API, or application programming interface, is a manner in which companies and organizations, like Twitter or the New York City government, expose their data and/or functionality to the public (i.e. talented programmers like yourself) for use. APIs allow us to add important data and functionality to the applications we build. Here's just a few examples of some of the cool things you can do by using APIs:

- Create an app that allows users to sign up/sign in via Facebook/Google/Twitter/Github/etc.
- Use the NYC Open Data API to get and map data—everything from Health Department restaurant ratings to public park locations and hours to New York City Public Housing repair issues to noise complaints to public school construction, you name it!

- Use the Yelp API to find and deliver popular local spots to your users.
- Use the Weather Underground API to give your users up-to-date weather alerts.
- Use the Ticket Master API to inform your users if their favorite musician has an upcoming show.
- So much more!

This is just a small sample of what working with APIs allows us to do as developers. Throughout the course of your programming life, you'll likely be exposed to working with many different APIs. You'll even learn how to build your own API later on in this course. This reading seeks to introduce the topic, emphasize some of the benefits of getting comfortable working with APIs and offer a brief intro into some of the common methods of working with APIs.

How to Work with APIs

Different APIs expose their data and functionalities in different ways. However, there are commonalities among them and there are common approaches that we'll discuss here. Generally speaking, there are two main uses for APIs—getting data and adding functionality (i.e. signing in with Facebook or posting to Instagram). We'll be discussing the "getting data" part of working with APIs here.

Many APIs are built on what is referred to as a REST-ful framework. That means that the "endpoints", or URLs to which we can send a request for data, follow certain conventions. These URLs should allow you to request information, send information, update information and delete information. Let's focus on the "getting information" request.

Retrieving Data from an API

For this walk-through, we'll be working with the Open Library API.

Let's say we want to build an app where users can search for books and look up information on them. We can make a custom request to the Open Library API for the exact information we want to present to the user.

Check out the [Open Library API documentation](https://openlibrary.org/dev/docs/api/search)  (<https://openlibrary.org/dev/docs/api/search>). We will be using the [search API](https://openlibrary.org/dev/docs/api/search)  (<https://openlibrary.org/dev/docs/api/search>).

Finding the API Endpoint

Take a few minutes and familiarize yourself with the docs linked above. Learning how to read API documentation is an important skill. It's okay if you don't understand everything in the above document. For now, just look it over briefly.

As you read through, you'll see that the documentation lists the API endpoint. **Endpoint** refers to the URL we can submit a request to and that will return to us the desired data.

Open up a new tab in your browser and paste in the following URL: <https://openlibrary.org/search.json?title=the+lord+of+the+rings>

The page brings you to is the desired set of data! Notice that the data is laid out in what looks like a big list of nested dictionaries. This is actually a [JSON](http://json.org/)  (<http://json.org/>) object, which behaves just like a Python dictionary. Working with the JSON data returned to you by requests to an API is one of the reasons why we spent so much time in previous lessons learning how to manipulate and operate on nested hashes.

For our app, we would like to show selected fields and a single book to our users. We can use [query parameters](#)  (<https://www.semrush.com/blog/url-parameters/>) to control the response format. The above URL already has a query parameter called `title`. We can add a `&` after the current URI to add more query parameters.

Open up a new tab again in your browser and paste in the following URL: https://openlibrary.org/search.json?title=the+lord+of+the+rings&fields=title,author_name&limit=1

The above URL should return a much more manageable response. It should look like the following:

```
{
  "numFound": 522,
  "start": 0,
  "numFoundExact": true,
  "docs": [
    {
      "title": "The Lord of the Rings",
      "author_name": ["J.R.R. Tolkien"]
    }
  ],
  "num_found": 522,
  "q": ""
```

```
"offset": null  
}
```

Top Tip: Once you find the right URL for retrieving your data, test it out directly in your browser *before* you try to request the data from inside your program. If pasting the URL into your browser brings you to the right data, you can move on. This cuts down on debugging time. This way, once you try to request the data from within your program, if it doesn't work, at least you'll know it's something wrong with your code, as opposed to something wrong with the API.

Sending a Request to an API from a Program or Application

Now that we understand what an API is and have even dealt with a URL that takes us to a real API endpoint, let's use that same URL to send a request for data from a Python program. Fork and clone this repo locally.

Open up `lib/open_library_api.py` . Let's take a look at the code here:

```
import requests  
import json
```

```
class Search:
```

```
    def get_search_results(self):  
        search_term = "the lord of the rings"
```

```
        search_term_formatted = search_term.replace(" ", "+")  
        fields = ["title", "author_name"]  
        # formats the list into a comma separated string  
        # output: "title,author_name"  
        fields_formatted = ",".join(fields)  
        limit = 1
```

```
        URL = f"https://openlibrary.org/search.json?title={search_term_formatted}&fields={fields_formatted}&limit={limit}"
```

```

        response = requests.get(URL)
        return response.content

    results = Search().get_search_results()

```

The `requests` library allows your program or application to send HTTP requests. In this case, we're using the `get` method with the `URL` argument to retrieve data from the Open Library API.

We define a `get_search_results()` method that assigns our API endpoint to a variable name `URL`. The method submits a request with that URL using the `get()` method defined in the `requests` library, and returns the content of the response.

Now, in your terminal in the directory of this lab, run `python lib/open_library_api.py`. It should output the response from the NYC Open Data API in a plaintext format:

```

b'{"numFound": 522, "start": 0, "numFoundExact": true, "docs": [{"title

```

Working with API Data

Let's write a method called `get_search_results_json` that returns the response formatted as JSON.

Copy and paste the following code into our Search class:

```

def get_search_results_json(self):
    search_term = "the lord of the rings"

    search_term_formatted = search_term.replace(" ", "+")
    fields = ["title", "author_name"]
    fields_formatted = ",".join(fields)
    limit = 1

    URL = f"https://openlibrary.org/search.json?title={search_term_formatted}&fields={fields_formatted}&limit={limit}

```

```
print(URL)
response = requests.get(URL)
```

At the bottom of the file, comment out:

```
results_json = Search().get_search_results_json()
# json.dumps formats the JSON object in a human readable format
print(json.dumps(results_json, indent=1))
```

Now, run the program with `python lib/open_library_api.py` in your terminal. You should see something like this:

```
{
  "numFound": 522,
  "start": 0,
  "numFoundExact": true,
  "docs": [
    {
      "title": "The Lord of the Rings",
      "author_name": ["J.R.R. Tolkien"]
    }
  ],
  "num_found": 522,
  "q": "",
  "offset": null
}
```

We have also provided a `get_user_search_results` method so you can experiment with using user input to create dynamic queries.

Solution

The final version of `open_library_api.py` :

```
import requests
import json

class Search:

    def get_search_results(self):
        search_term = "the lord of the rings"

        search_term_formatted = search_term.replace(" ", "+")
        fields = ["title", "author_name"]
        # formats the list into a comma separated string
        # output: "title,author_name"
        fields_formatted = ",".join(fields)
        limit = 1

        URL = f"https://openlibrary.org/search.json?title={search_term_formatted}&fields={fields_formatted}&limit={limit}"

        response = requests.get(URL)
        return response.content

    def get_search_results_json(self):
        search_term = "the lord of the rings"

        search_term_formatted = search_term.replace(" ", "+")
        fields = ["title", "author_name"]
        fields_formatted = ",".join(fields)
        limit = 1

        URL = f"https://openlibrary.org/search.json?title={search_term_formatted}&fields={fields_formatted}&limit={limit}"
        print(URL)
```

```
response = requests.get(URL)
return response.json()

def get_user_search_results(self, search_term):
    search_term_formatted = search_term.replace(" ", "+")
    fields = ["title", "author_name"]
    fields_formatted = ",".join(fields)
    limit = 1

    URL = f"https://openlibrary.org/search.json?title={search_term_formatted}&fields={fields_formatted}&limit={limit}"

    response = requests.get(URL).json()
    response_formatted = f>Title: {response['docs'][0]['title']}\nAuthor: {response['docs'][0]['author_name'][0]}
    return response_formatted

# results = Search().get_search_results()
# print(results)

# results_json = Search().get_search_results_json()
# print(json.dumps(results_json, indent=1))

search_term = input("Enter a book title: ")
result = Search().get_user_search_results(search_term)
print("Search Result:\n")
print(result)
```

Conclusion

To recap: APIs generally either provide a user with data or added functionality. We can use APIs that serve data to get information for our own applications and projects. To get this data, we need to send a request to the URL of the API and know how to work with the response we

receive. Many APIs serve data in JSON format, which needs to be parsed before we can use it. Once parsed, it becomes a dictionary we can work with and extract data from.

In our example, we were able to retrieve remote information from an API using the `requests` library. By putting this implementation inside a class, we can develop highly reusable code that lets us access all sorts of information remotely.

Always make sure to read the API docs and experiment with queries in the browser or in the Python REPL before writing out custom classes.

Resources

- [GET - Mozilla](https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/GET)  [_ \(https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/GET\)](https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods/GET)
- [HTTP methods - Mozilla](https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods)  [_ \(https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods\)](https://developer.mozilla.org/en-US/docs/Web/HTTP/Methods)
- [requests](https://requests.readthedocs.io/en/latest/)  [_ \(https://requests.readthedocs.io/en/latest/\)](https://requests.readthedocs.io/en/latest/)
- [Python JSON](https://docs.python.org/3/library/json.html)  [_ \(https://docs.python.org/3/library/json.html\)](https://docs.python.org/3/library/json.html)

This tool needs to be loaded in a new browser window

Load Working with APIs (Updated) (CodeGrade) in a new window